# Blockchain Experiment 3

**Name:** Atharva Lotankar
**Class**: D20C **R.No.:** 20
**Date:** 26 / 08 / 2026

**Aim:** Create a Cryptocurrency using Python and perform mining in the Blockchain created.

**Theory:**

### 1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks. Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

### 2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
   Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

### 3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (`5001`, `5002`, `5003`). Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

### 4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

### 5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

### 6. Chain Replacement

When `/replace_chain` is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

### Tools & Libraries Used

- **Python 3.x**
- **Flask** – Web framework for API endpoints

  ```
  pip install Flask
  ```

- **Requests** – For HTTP communication between nodes

  ```
  pip install requests==2.18.4
  ```

- **Postman** – For testing API requests
- Python Standard Libraries:
  - `datetime`
  - `jsonify`
  - `hashlib`
  - `uuid4`
  - `urlparse`
  - `request`

**Procedure & Output:**

**Step 1:** Download hadcoin_node_5001.py, adding transaction in mine function, following copy hadcoin_node_5002.py and hadcoin_node_5003.py, with changing their port number. Run each node in separate terminals.

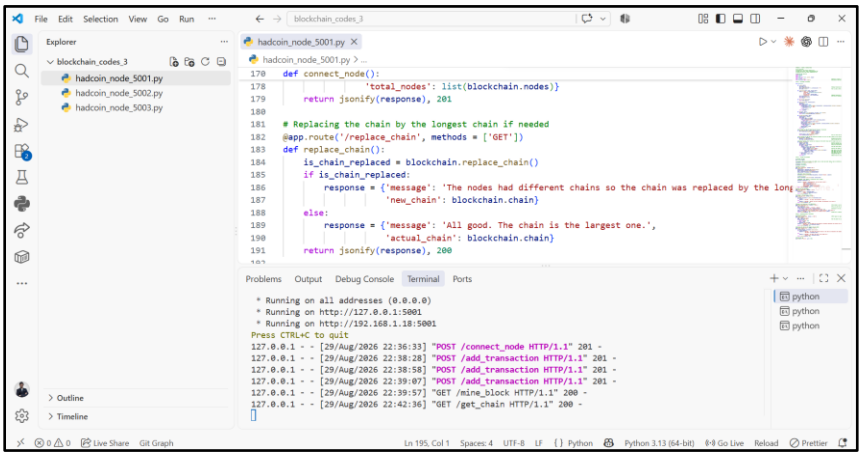
### #Program of hadcoin_node

```
# Importing the libraries
import datetime
import hashlib
import json
from flask import Flask, jsonify, request
import requests
from uuid import uuid4                                  # Generate a unique id that is in hex
from urllib.parse import urlparse                       # To parse url of the nodes

# Part 1 - Building a Blockchain

class Blockchain:

    def __init__(self):
        self.chain = []
        self.transactions = []          # Adding transactions before they are added to a block
        self.create_block(proof = 1, previous_hash = '0')
        self.nodes = set()
    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                 'timestamp': str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash': previous_hash,
                 'transactions': self.transactions}
self.transactions = []
self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
            hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys = True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
```

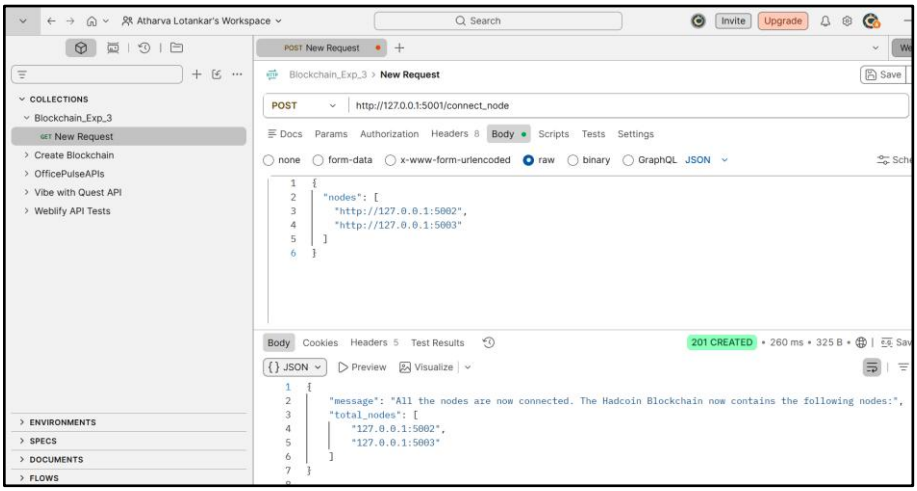
```
            block = chain[block_index]
            if block['previous_hash'] != self.hash(previous_block):
                return False
            previous_proof = previous_block['proof']
            proof = block['proof']
            hash_operation = hashlib.sha256(str(proof**2 -
previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] != '0000':
                return False
            previous_block = block
            block_index += 1
        return True

    # This method will add the trqnsaction to the list of trnsactions
    def add_transaction(self, sender, receiver, amount):
        self.transactions.append({'sender': sender,
                                  'receiver': receiver,
                                  'amount': amount})
        previous_block = self.get_previous_block()
        return previous_block['index'] + 1
    # This function will add the node containing an address to the set of nodes created in init
function
    def add_node(self, address):
        parsed_url = urlparse(address)
self.nodes.add(parsed_url.netloc)
    # Consensus Protocol. This function will replace all the shorter chain with the longer chain
in all the nodes on the network
    def replace_chain(self):
        network = self.nodes
longest_chain = None                                                              max_length =
len(self.chain)                                                  for node in network:
            response = requests.get(f'http://{node}/get_chain')                           if
response.status_code == 200:
                length = response.json()['length']
chain = response.json()['chain']
                if length > max_length and self.is_chain_valid(chain):
max_length = length
if longest_chain:
            self.chain = longest_chain
return True
        return False
# Part 2 - Mining our Blockchain

# Creating a Web App
app = Flask(__name__)

# Creating an address for the node on Port 5000. We will create some other nodes as well on
different ports
node_address = str(uuid4()).replace('-', '')

# Creating a Blockchain
blockchain = Blockchain()

# Mining a new block
@app.route('/mine_block', methods=['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)

    # Add block reward transaction
```

```
    blockchain.add_transaction(sender=node_address, receiver='Richard', amount=1)

    # Create the block
    block = blockchain.create_block(proof, previous_hash)

    # IMPORTANT: Clear mempool so transactions aren't remined
    blockchain.transactions = []

    response = {'message': 'Congratulations, you just mined a block!',
                'index': block['index'],
                'timestamp': block['timestamp'],
                'proof': block['proof'],
                'previous_hash': block['previous_hash'],
                'transactions': block['transactions']}
    return jsonify(response), 200

# Getting the full Blockchain
@app.route('/get_chain', methods = ['GET'])
def get_chain():
    response = {'chain': blockchain.chain,
                'length': len(blockchain.chain)}
    return jsonify(response), 200

# Checking if the Blockchain is valid
@app.route('/is_valid', methods = ['GET'])
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
    return jsonify(response), 200

# Adding a new transaction to the Blockchain
@app.route('/add_transaction', methods = ['POST'])                              # Post method as
we have to pass something to get something in return
def add_transaction():
    json = request.get_json()                                                   # This will get
the json file from postman. In Postman we will create a json file in which we will pass the
values for the keys in the json file
    transaction_keys = ['sender', 'receiver', 'amount']
    if not all(key in json for key in transaction_keys):                        # Checking if all
keys are available in json
        return 'Some elements of the transaction are missing', 400
    index = blockchain.add_transaction(json['sender'], json['receiver'], json['amount'])
    response = {'message': f'This transaction will be added to Block {index}'}
    return jsonify(response), 201                                               # Code 201 for
creation

# Part 3 - Decentralizing our Blockchain

# Connecting new nodes
@app.route('/connect_node', methods = ['POST'])                                 # POST request to
register the new nodes from the json file
def connect_node():
    json = request.get_json()
    nodes = json.get('nodes')                                                   # Get the nodes
from json file
    if nodes is None:
        return "No node", 400
    for node in nodes:
        blockchain.add_node(node)
```

```
    response = {'message': 'All the nodes are now connected. The Hadcoin Blockchain now contains
the following nodes:',
                'total_nodes': list(blockchain.nodes)}
    return jsonify(response), 201

# Replacing the chain by the longest chain if needed
@app.route('/replace_chain', methods = ['GET'])
def replace_chain():
    is_chain_replaced = blockchain.replace_chain()
    if is_chain_replaced:
        response = {'message': 'The nodes had different chains so the chain was replaced by the
longest one.',
                    'new_chain': blockchain.chain}
    else:
        response = {'message': 'All good. The chain is the largest one.',
                    'actual_chain': blockchain.chain}
    return jsonify(response), 200

# Running the app
app.run(host = '0.0.0.0', port = 5001)
```

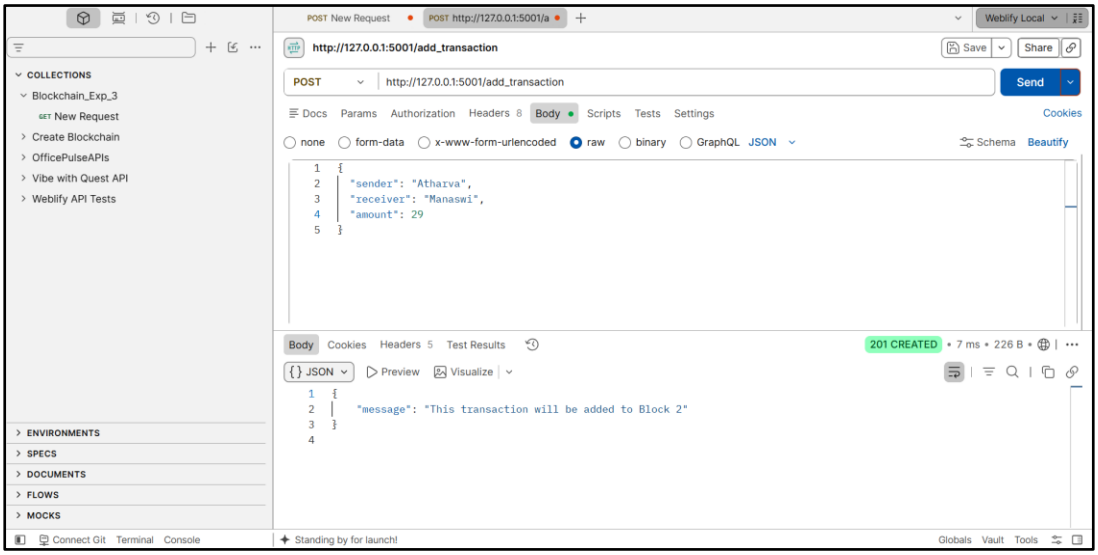


**Step 2:** Connected Node `5001` to its peer nodes (`[http://127.0.0.1:5002](http://127.0.0.1:5002)` and `[http://127.0.0.1:5003](http://127.0.0.1:5003)`) by invoking the `/connect_node` endpoint via a `POST` request containing the peer addresses in JSON format.



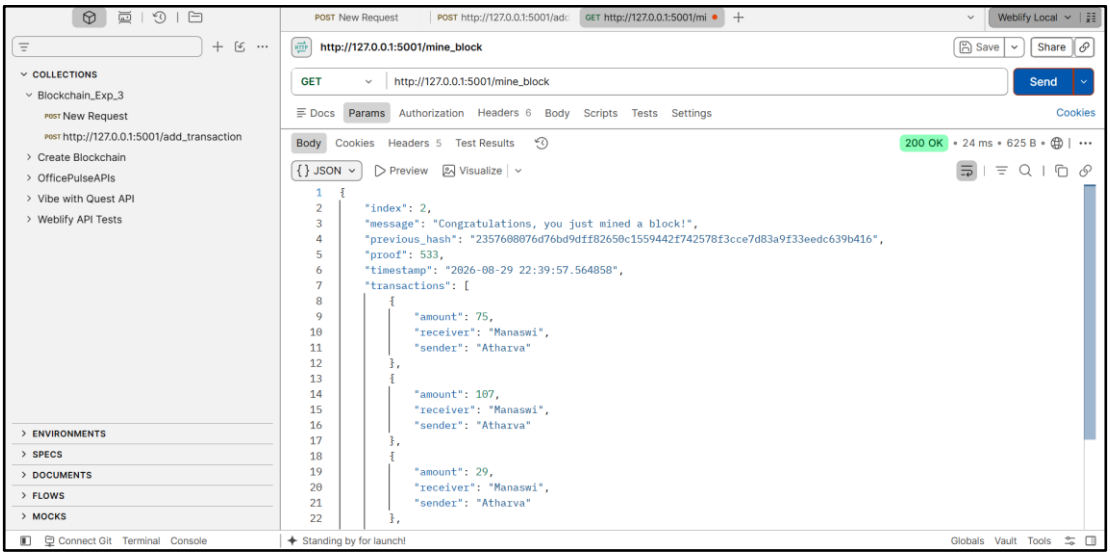
*Connecting Node 5001 to Node 5002 and Node 5003 via `POST /connect_node`*

**Step 3:** Adding Pending Transactions to the Mempool Submitted transactions (`sender: "Atharva"`, `receiver: "Manaswi"`) to Node 5001 using the `/add_transaction` endpoint. The transactions were queued in Node 5001's mempool awaiting mining.



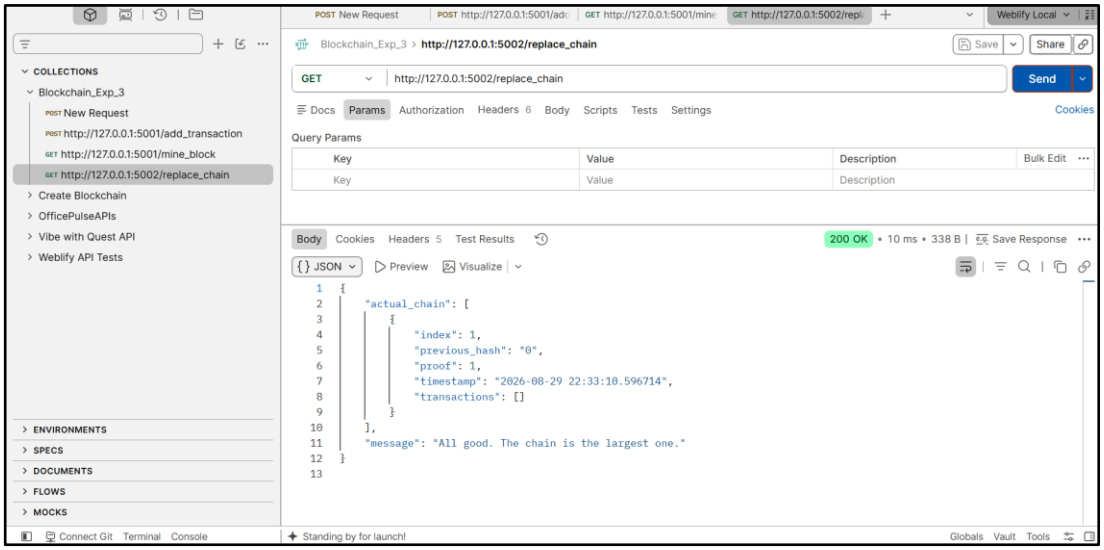

*Adding a transaction to Node 5001's mempool via `POST /add_transaction`*

**Step 4:** Executed the Proof-of-Work mining algorithm on Node 5001 by calling `GET /mine_block`. The pending transactions were bundled into Block 2, a mining reward transaction was appended, and the mempool was cleared.



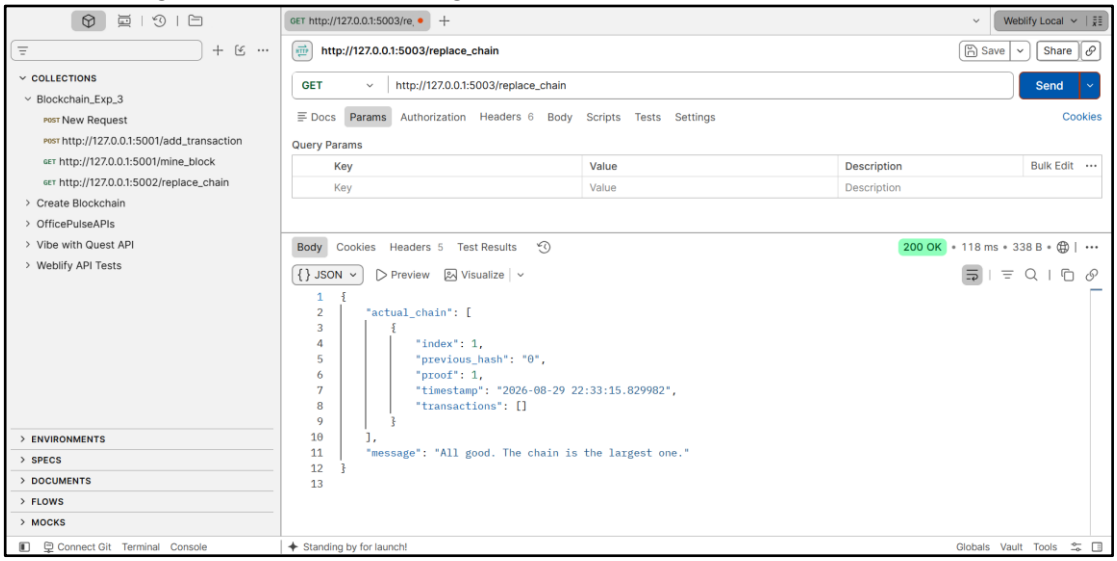

*Mining Block 2 on Node 5001 containing queued transactions via `GET /mine_block`*

**Step 5:** Triggered the consensus algorithm on Node 5002 using `GET /replace_chain` to synchronize its ledger with Node 5001's longer chain.



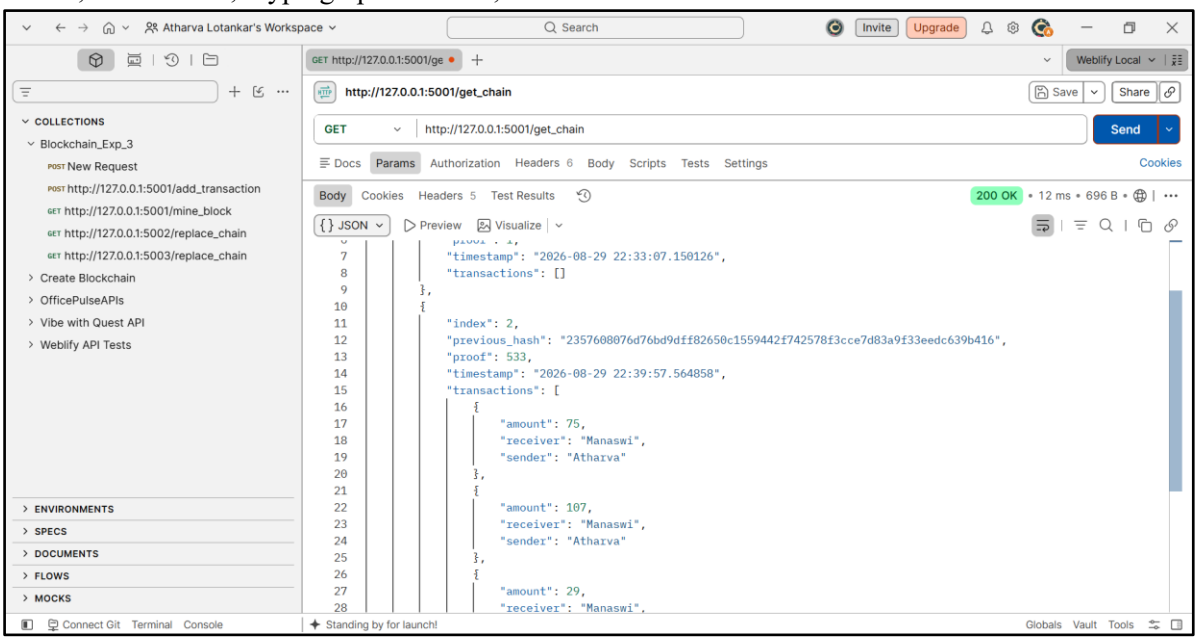

*Executing consensus on Node 5002 via `GET /replace_chain`*

**Step 6:** Triggered the consensus algorithm on Node 5003 using `GET /replace_chain` to synchronize its ledger with Node 5001's longer chain.




*Executing consensus on Node 5003 via `GET /replace_chain`*

**Step 7:** Retrieved the full blockchain state from Node 5001 using `GET /get_chain` to verify the structure, block index, cryptographic hashes, and confirmed transactions across the network.




*Fetching the updated full blockchain via `GET /get_chain`*

## Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.